

```

---
title: Traces
weight: 1
description: The path of a request through your application.
cSpell:ignore: Guten
---
```

****Traces**** give us the big picture of what happens when a request is made to an application. Whether your application is a monolith with a single database or a sophisticated mesh of services, traces are essential to understanding the full "path" a request takes in your application.

Let's explore this with three units of work, represented as [Spans](#spans):

```

> [!NOTE]
>
> The following JSON examples do not represent a specific format, and especially
> not [OTLP/JSON](/docs/specs/otlp/#json-protobuf-encoding), which is more
> verbose.
```

`hello` span:

```

```json
{
 "name": "hello",
 "context": {
 "trace_id": "5b8aa5a2d2c872e8321cf37308d69df2",
 "span_id": "051581bf3cb55c13"
 },
 "parent_id": null,
 "start_time": "2022-04-29T18:52:58.114201Z",
 "end_time": "2022-04-29T18:52:58.114687Z",
 "attributes": {
 "http.route": "some_route1"
 },
 "events": [
 {
 "name": "Guten Tag!",
 "timestamp": "2022-04-29T18:52:58.114561Z",
 "attributes": {
 "event_attributes": 1
 }
 }
]
}
```
```

This is the root span, denoting the beginning and end of the entire operation. Note that it has a `trace\_id` field indicating the trace, but has no `parent\_id`. That's how you know it's the root span.

`hello-greetings` span:

```

```json
{
 "name": "hello-greetings",
 "context": {
 "trace_id": "5b8aa5a2d2c872e8321cf37308d69df2",
 "span_id": "5fb397be34d26b51"
 },
 "parent_id": "051581bf3cb55c13",
 "start_time": "2022-04-29T18:52:58.114304Z",
 "end_time": "2022-04-29T22:52:58.114561Z",
 "attributes": {
 "http.route": "some_route2"
 },
 "events": [
 {
 "name": "hey there!",
 "timestamp": "2022-04-29T18:52:58.114561Z",
 "attributes": {
 "event_attributes": 1
 }
 }
],
 {
 "name": "bye now!",
```

```

 "timestamp": "2022-04-29T18:52:58.114585Z",
 "attributes": {
 "event_attributes": 1
 }
 }
]
}
...

```

This span encapsulates specific tasks, like saying greetings, and its parent is the `hello` span. Note that it shares the same `trace\_id` as the root span, indicating it's a part of the same trace. Additionally, it has a `parent\_id` that matches the `span\_id` of the `hello` span.

`hello-salutations` span:

```

```json
{
  "name": "hello-salutations",
  "context": {
    "trace_id": "5b8aa5a2d2c872e8321cf37308d69df2",
    "span_id": "93564f51e1abe1c2"
  },
  "parent_id": "051581bf3cb55c13",
  "start_time": "2022-04-29T18:52:58.114492Z",
  "end_time": "2022-04-29T18:52:58.114631Z",
  "attributes": {
    "http.route": "some_route3"
  },
  "events": [
    {
      "name": "hey there!",
      "timestamp": "2022-04-29T18:52:58.114561Z",
      "attributes": {
        "event_attributes": 1
      }
    }
  ]
}
...

```

This span represents the third operation in this trace and, like the previous one, it's a child of the `hello` span. That also makes it a sibling of the `hello-greetings` span.

These three blocks of JSON all share the same `trace\_id`, and the `parent\_id` field represents a hierarchy. That makes it a Trace!

Another thing you'll note is that each Span looks like a structured log. That's because it kind of is! One way to think of Traces is that they're a collection of structured logs with context, correlation, hierarchy, and more baked in. However, these "structured logs" can come from different processes, services, VMs, data centers, and so on. This is what allows tracing to represent an end-to-end view of any system.

To understand how tracing in OpenTelemetry works, let's look at a list of components that will play a part in instrumenting our code.

Tracer Provider

A Tracer Provider (sometimes called `TracerProvider`) is a factory for `Tracer`s. In most applications, a Tracer Provider is initialized once and its lifecycle matches the application's lifecycle. Tracer Provider initialization also includes Resource and Exporter initialization. It is typically the first step in tracing with OpenTelemetry. In some language SDKs, a global Tracer Provider is already initialized for you.

Tracer

A Tracer creates spans containing more information about what is happening for a given operation, such as a request in a service. Tracers are created from Tracer Providers.

Trace Exporters

Trace Exporters send traces to a consumer. This consumer can be standard output

for debugging and development-time, the OpenTelemetry Collector, or any open source or vendor backend of your choice.

Context Propagation

Context Propagation is the core concept that enables Distributed Tracing. With Context Propagation, Spans can be correlated with each other and assembled into a trace, regardless of where Spans are generated. To learn more about this topic, see the concept page on [Context Propagation](../../context-propagation).

Spans

A **span** represents a unit of work or operation. Spans are the building blocks of Traces. In OpenTelemetry, they include the following information:

- Name
- Parent span ID (empty for root spans)
- Start and End Timestamps
- [Span Context](#span-context)
- [Attributes](#attributes)
- [Span Events](#span-events)
- [Span Links](#span-links)
- [Span Status](#span-status)

Sample span:

```
```json
{
 "name": "/v1/sys/health",
 "context": {
 "trace_id": "7bba9f33312b3dbb8b2c2c62bb7abe2d",
 "span_id": "086e83747d0e381e"
 },
 "parent_id": "",
 "start_time": "2021-10-22 16:04:01.209458162 +0000 UTC",
 "end_time": "2021-10-22 16:04:01.209514132 +0000 UTC",
 "status_code": "STATUS_CODE_OK",
 "status_message": "",
 "attributes": {
 "net.transport": "IP.TCP",
 "net.peer.ip": "172.17.0.1",
 "net.peer.port": "51820",
 "net.host.ip": "10.177.2.152",
 "net.host.port": "26040",
 "http.method": "GET",
 "http.target": "/v1/sys/health",
 "http.server_name": "mortar-gateway",
 "http.route": "/v1/sys/health",
 "http.user_agent": "Consul Health Check",
 "http.scheme": "http",
 "http.host": "10.177.2.152:26040",
 "http.flavor": "1.1"
 },
 "events": [
 {
 "name": "",
 "message": "OK",
 "timestamp": "2021-10-22 16:04:01.209512872 +0000 UTC"
 }
]
}
```
```

Spans can be nested, as is implied by the presence of a parent span ID: child spans represent sub-operations. This allows spans to more accurately capture the work done in an application.

Span Context

Span context is an immutable object on every span that contains the following:

- The Trace ID representing the trace that the span is a part of
- The span's Span ID
- Trace Flags, a binary encoding containing information about the trace
- Trace State, a list of key-value pairs that can carry vendor-specific trace information

Span context is the part of a span that is serialized and propagated alongside [Distributed Context](#context-propagation) and [Baggage](../baggage).

Because Span Context contains the Trace ID, it is used when creating [Span Links](#span-links).

Attributes

Attributes are key-value pairs that contain metadata that you can use to annotate a Span to carry information about the operation it is tracking.

For example, if a span tracks an operation that adds an item to a user's shopping cart in an eCommerce system, you can capture the user's ID, the ID of the item to add to the cart, and the cart ID.

You can add attributes to spans during or after span creation. Prefer adding attributes at span creation to make the attributes available to SDK sampling. If you have to add a value after span creation, update the span with the value.

Attributes have the following rules that each language SDK implements:

- Keys must be non-null string values
- Values must be a non-null string, boolean, floating point value, integer, or an array of these values

Additionally, there are [Semantic Attributes](/docs/specs/semconv/general/trace/), which are known naming conventions for metadata that is typically present in common operations. It's helpful to use semantic attribute naming wherever possible so that common kinds of metadata are standardized across systems.

Span Events

A Span Event can be thought of as a structured log message (or annotation) on a Span, typically used to denote a meaningful, singular point in time during the Span's duration.

For example, consider two scenarios in a web browser:

1. Tracking a page load
2. Denoting when a page becomes interactive

A Span is best used to track the first scenario because it's an operation with a start and an end.

A Span Event is best used to track the second scenario because it represents a meaningful, singular point in time.

When to use span events versus span attributes

Since span events also contain attributes, the question of when to use events instead of attributes might not always have an obvious answer. To inform your decision, consider whether a specific timestamp is meaningful.

For example, when you're tracking an operation with a span and the operation completes, you might want to add data from the operation to your telemetry.

- If the timestamp in which the operation completes is meaningful or relevant, attach the data to a span event.
- If the timestamp isn't meaningful, attach the data as span attributes.

Span Links

Links exist so that you can associate one span with one or more spans, implying a causal relationship. For example, let's say we have a distributed system where some operations are tracked by a trace.

In response to some of these operations, an additional operation is queued to be executed, but its execution is asynchronous. We can track this subsequent operation with a trace as well.

We would like to associate the trace for the subsequent operations with the first trace, but we cannot predict when the subsequent operations will start. We need to associate these two traces, so we will use a span link.

You can link the last span from the first trace to the first span in the second trace. Now, they are causally associated with one another.

Links are optional but serve as a good way to associate trace spans with one another.

For more information see [\[Span Links\]\(/docs/specs/otel/trace/api/#link\)](#).

Span Status

Each span has a status. The three possible values are:

- ``Unset``
- ``Error``
- ``Ok``

The default value is ``Unset``. A span status that is ``Unset`` means that the operation it tracked successfully completed without an error.

When a span status is ``Error``, then that means some error occurred in the operation it tracks. For example, this could be due to an HTTP 500 error on a server handling a request.

When a span status is ``Ok``, then that means the span was explicitly marked as error-free by the developer of an application. Although this is unintuitive, it's not required to set a span status as ``Ok`` when a span is known to have completed without error, as this is covered by ``Unset``. What ``Ok`` does is represent an unambiguous "final call" on the status of a span that has been explicitly set by a user. This is helpful in any situation where a developer wishes for there to be no other interpretation of a span other than "successful".

To reiterate: ``Unset`` represents a span that completed without an error. ``Ok`` represents when a developer explicitly marks a span as successful. In most cases, it is not necessary to explicitly mark a span as ``Ok``.

Span Kind

When a span is created, it is one of ``Client``, ``Server``, ``Internal``, ``Producer``, or ``Consumer``. This span kind provides a hint to the tracing backend as to how the trace should be assembled. According to the OpenTelemetry specification, the parent of a server span is often a remote client span, and the child of a client span is usually a server span. Similarly, the parent of a consumer span is always a producer and the child of a producer span is always a consumer. If not provided, the span kind is assumed to be internal.

For more information regarding `SpanKind`, see [\[SpanKind\]\(/docs/specs/otel/trace/api/#spankind\)](#).

Client

A client span represents a synchronous outgoing remote call such as an outgoing HTTP request or database call. Note that in this context, "synchronous" does not refer to ``async/await``, but to the fact that it is not queued for later processing.

Server

A server span represents a synchronous incoming remote call such as an incoming HTTP request or remote procedure call.

Internal

Internal spans represent operations which do not cross a process boundary. Things like instrumenting a function call or an Express middleware may use internal spans.

Producer

Producer spans represent the creation of a job which may be asynchronously processed later. It may be a remote job such as one inserted into a job queue or a local job handled by an event listener.

Consumer

Consumer spans represent the processing of a job created by a producer and may

start long after the producer span has already ended.

Specification

For more information, see the
[traces specification](/docs/specs/otel/overview/#tracing-signal).